Lotka-Volterra

Simone Valigi

A.A. 2025-2026
Programmazione per la Fisica

Indice

1 Progetto **2**
1.1 Descrizione del sistema 2
1.2 Dinamica del sistema 2
1.3 Obiettivi del codice 3

2 Implementazione **3**
2.1 Struttura generale del codice 3
2.2 Dettagli di implementazione 3
2.3 Repository su github 5

3 Logica dei test **5**

4 Analisi dei risultati **6**

5 Compilazione del progetto **6**
5.1 Struttura 6
5.2 Compilazione del codice sorgente 6
5.2.1 Tramite CMake 6
5.2.2 Tramite g++ 7
5.3 Esecuzione del Programma Principale e dei Test Unitari 7

1 Progetto

Lo scopo del progetto è la costruzione di un programma `C++` che simuli l'interazione tra due specie animali secondo il Modello Lotka-Volterra. Le due specie sono identificate tramite due variabili: x e y, rispettivamente per le prede e i predatori. Gli ultimi si cibano dei primi.

1.1 Descrizione del sistema

Identificate le popolazioni di prede e predatori ad un certo istante temporale come $x(t)$ e $y(t)$, ne modelliamo l'andamento temporale tramite il set di equazioni differenziali, dette Equazioni di Lotka-Volterra:

$$\frac{\mathrm{d}x}{\mathrm{dt}} = [A - By(t)]\, x(t) \qquad \frac{\mathrm{d}y}{\mathrm{dt}} = [Cx(t) - D]\, y(t)$$

dove i coefficienti rappresentano

- A - tasso di riproduzione delle prede
- B - tasso di mortalità delle prede
- C - tasso di riproduzione dei predatori
- D - tasso di mortalità dei predatori

La riproduzione è data dal consumo delle risorse disponibili alle due specie. La mortalità delle prede è data dall'interazione con i predatori, che ne causano la morte; analogamente la riproduzione dei predatori è data dalla disponibilità di prede nel sistema e la mortalità dalla scarsità di prede. Tutti questi parametri devono essere considerati maggiori di zero.

Durante la simulazione viene usata la versione discretizzata delle equazioni, tramite il *metodo di Eulero simplettico*:

$$x_i = x_{i-1} + (A - By_i)\, x_{i-1}\Delta t \qquad y_i = y_{i-1} + (Cx_{i-1} - D)\, y_{i-1}\Delta t$$

La scelta di moltiplicare B per y_i nel calcolo di x_i si discosta dal metodo di integrazione che invece prevede y_{i-1}. Questa scelta garantisce una stabilità maggiore del calcolo dell'integrale primo del moto $H(x, y)$, ma impone l'accorgimento di dover calcolare ad ogni passo dell'integrazione numerica prima y_i e poi x_i.

1.2 Dinamica del sistema

Consideriamo uno stato iniziale determinato da (x_0, y_0), tali che siano entrambi maggiori di 0, allora le orbite del sistame sono ristrette a valori positivi di $x(t)$ e $y(t)$. Questa richiesta è giustificata dal senso fisico del sistema: un numero di individui negativo non ha senso di esistere.

Risolvendo il sistema di equazioni differenziali si ottengo due punti di equilibrio:

$$e_1 = (0, 0) \quad \text{e} \quad e_2 = \left(\frac{D}{C}, \frac{A}{B}\right)$$

che sono, rispettivamente, il caso con assenza contemporanea di prede e predatori e il secondo è l'unico caso non banale in cui l'evoluzione del sistema garantisce la stazionarietà delle popolazioni: si tratta di un perfetto equilibrio tra tutti i parametri del sistema.

Inoltre, si può ottenere una funzione che assume il ruolo di "energia" del sistema, per cui scelte delle condizioni si conserva lungo questa traiettoria. Per questa similitudine, verrà indicata con H:

$$H(x, y) = Cx + By - [D \ln(x) + A \ln(y)]$$

In particolare il punto di equilibrio $\left(\frac{D}{C}, \frac{A}{B}\right)$ rappresenta anche il punto di minimo di $H(x, y)$

1.3 Obiettivi del codice

Forniti in input i valori iniziali (x_0, y_0) e i quattro parametri delle equazioni di Lotka-Volterra, il programma sviluppa una simulazione che ad ogni step temporale calcola la terna (x_i, y_i, H_i) e la salvi su un file `.txt`.

2 Implementazione

2.1 Struttura generale del codice

Il programma è organizzato in due cartelle `source` e `test`:

- `simulation.cpp` e il suo header file `simulation.hpp`, in `source`, descrivono la classe `Simulation`, che rappresenta al suo interno una singola simulazione e ne mantinene internamente lo stato. Inoltre, sono presenti i metodi per fare evolvere la simulazione di un *time step* e visualizzarne i risultati.
- `test_simulation.cpp`, in `test`, contiene i test per testare il funzionamento della classe.
- `main.cpp`, in `source`, è l'*entry point* del programma: crea l'oggetto `Simulation`, evolve di un time step la simulazione, stampa schermo i risultati e li salva in un file.

2.2 Dettagli di implementazione

La classe `Simulation`, definita nell'header file e implementata in `simulation.cpp`, lo scheletro del programma: gestisce la simulazione dell'interazione e contiene i seguenti attributi e metodi principali:

Attributi:

- `double x_rel, y_rel`: numeri, relativi al punto di equilibrio, correnti di prede e predatori
- `double A, B, C, D`: parametri delle equazioni di Lotka-Volterra, come input
- `double x_eq, y_eq`: numeri di entità al punto di equilibrio
- `double T, dT`: tempo corrente e time step della simulazione

Metodi:

- `Simulation(simul_params const& params)`: costruttore che inizializza la simulazione attraverso i parametri di input; i parametri passati sono raggruppati nella `struct` chiamata `simul_params`
- `double get_relative_x(), get_relative_y()`: ottengono i valori relativi delle popolazioni correnti
- `double get_absolute_x(), get_absolute_y()`: calcolano i valori assoluti delle popolazioni correnti
- `double get_x_eq(), get_y_eq()`: ottengono i valori delle popolazioni all'equilibrio
- `double get_time()`: ottiene il tempo corrente
- `double get_H()`: calcola il valore dell'integrale del moto $H(x, y)$ corrente
- `void evolve()`: aggiorna i valori di `x_rel` e `y_rel`
- `std::string info()`: ritorna una stringa che rappresenta i valori (x_i, y_i, H_i) di interesse della simulazione

In particolare, il metodo `evolve()` è l'altro fulcro della simulazione poiché aggiorna lo stato delle popolazioni di prede e predatori ad ogni istante di tempo. Al suo interno, vengono usate le equazioni discretizzate di Lotka-Volterra.

```
    // time evolution method:
void Simulation::evolve() {
  T += dT;

  y_rel += D * (x_rel - 1.0) * y_rel * dT;
  x_rel += A * (1.0 - y_rel) * x_rel * dT;

  // ensuring positive new relative values:
  if (x_rel <= 0 || y_rel <= 0) {
    throw std::runtime_error(
        "Error: Invalid state occurred, x and y values must be positive");
```

```
  }

  // ensuring non-infinite new relative values:
  if (std::isinf(x_rel) || std::isinf(y_rel)) {
    throw std::runtime_error(
        "Error: Invalid state occurred, overflow during simulation");
  }
}
```

In `main.cpp` viene gestito l'input dell'utente dalla riga di comando. Infatti, vengono gestiti i casi in cui l'utente inserisce i parametri all'invocazione della funzione e il caso in cui viene invocata soltanto la funzione e un'altra funzione `void insert_params(simul_params& params)` gestisce l'interfaccia sul terminale; in entrambi i casi i parametri entrano nella `struct simul_params`. Inoltre, instanzia un oggetto `Simulation` che rappresenta lo stato iniziale delle popolazioni di prede e predatori e i parametri delle equazioni.

Non appena conclusa l'inizializzazione, la simulazione procede per un numero di time steps decisi dall'utente come un intero del passo temporale deciso, stampando a schermo i punti (x_i, y_i, H_i) calcolati e salvandoli in un file `simulation_output.txt`.

2.3 Repository su github

Il progetto è disponibile su GitHub, alla repo Lotka-Volterra del mio profilo simonevaligi.

3 Logica dei test

L'implementazione dei test unitari è stata realizzata tramite la libreria `doctest` e coprono molteplici aspetti del programma, per garantire un buon funzionamento dei casi critici o al limite. In particolare:

- controllo dei parametri iniziali della simulazione a prevenzione di valori non validi: valori corretti, valori nulli e valori negativi
- controllo del calcolo del punto di equilibrio, del calcolo dei valori relativi e assoluti
- controllo dell'evoluzione temporale delle popolazioni: incremento temporale, stabilità al punto di equilibrio, ottenimento dei valori assoluti e valutazione del segno e della magnitudo della simulazione
- controllo del calcolo dell'integrale del moto $H(x, y)$ e della sua conservazione
- controllo della stampa su schermo
- verifica della divergenza ad un passo temporale molto grande rispetto all'ordine di grandezza consigliato (~ 1 ms)

4 Analisi dei risultati

Si suggerisce una simulazione con i seguenti parametri all'invocazione:

```
initial_x = 1.8;
initial_y = 4.3;
A = 0.1;
B = 0.02;
C = 0.05;
D = 0.1;
dt = 0.001;
steps = 130000;
```

L'approssimazione armonica delle equazioni di Lotka-Volterra implica un periodo di oscillazione

$$T \simeq \frac{2\pi}{\sqrt{AD}}$$

per cui, per ottenere all'incirca due periodi (necessari a verificare soddisfacentemente la periodicità) si consiglia di valutare attentamente il numero di passi temporali inseriti tramite la formula riportata.

5 Compilazione del progetto

5.1 Struttura

Il progetto è strutturato con una directory principale in cui sono presenti

- le subdirectories
  - `source`: contiene i file sorgente del programma principale
  - `test`: contiene la libreria `doctest`, nella forma del file `doctest.h`
- i file
  - `CMakeLists.txt`: file di configurazione per CMake, utilizzato per la generazione dei Makefile
  - `.clang-format`: standard file di formattazione del codice generato dal comando
    `$ clang-format --sdump-config -style=google > .clang-format`

5.2 Compilazione del codice sorgente

5.2.1 Tramite CMake

Building:
Dopo aver verificato la presenza del *ninja build system* e la versione di CMake (necessaria la 3.28), ci si ponga nella directory principale e si eseguano:

```
$ cmake -S . -B build -G"Ninja Multi-Config"
```

Verranno creati dei file nella cartella `build`.
Compilazione:
Per compilare in modalità *Debug* e *Release* eseguire rispettivamente:

```
$ cmake --build build --config Debug
```

e

```
$ cmake --build build --config Release
```

Dentro le cartelle `./build/Debug` e `./build/Release` saranno presenti gli eseguibili `lotka_volterra` e `lotka_volterra.t`, relativo ai test.

5.2.2 Tramite g++

Sempre nella cartella madre, la compilazione può essere eseguita tramite

```
g++ -std=c++20 -Wall -Wextra source/main.cpp source/simulation.cpp -o lotka_volterra
```

che genererà l'eseguibile `lotka_volterra` nella cartella madre.
Per compilare i test:

```
g++ -std=c++20 test/test_simulation.cpp source/simulation.cpp -o test_simulation
```

che, analogamente, produrrà l'eseguibile dei test `test_simulation`.

5.3 Esecuzione del Programma Principale e dei Test Unitari

Chiamare i seguenti comandi per l'esecuzione dei test sia in modalità *Debug* che *Release*:

```
$ cmake --build build --config Debug --target test
```

e

```
$ cmake --build build --config Release --target test
```

Chiamare i seguenti comandi per l'esecuzione del programma principale (sia in modalità *Debug* che *Release*):

```
$ ./build/Debug/lotka_volterra
```

e

```
$ ./build/Release/lotka_volterra
```